B.Sc. (Data Science)

DS-151-T : Advanced Python Programming

Unit3: Exception Handling

Mrs. Reshma Masurekar
Dr. D. Y. Patil Arts, Commerce and Science College,
Pimpri, Pune 18

Unit3: Exception Handling

- 3.1 Python Exception
- 3.2 Common Exception
- 3.3 Exception handling in Python (try-except-else)
- 3.4 The except statement with no exception
- 3.5 Multiple Exception
- 3.6 The try-finally clause
- 3.7 Custom Exception and assert statement

Error Vs Exception

- **Errors** represent conditions such as compilation error, syntax error, error in the logical part of the code, library incompatibility, infinite recursion, etc.
- Errors are usually beyond the control of the programmer and we should not try to handle errors.
- **Exceptions** can be caught and handled by the program.

Python Exceptions

- When a Python program meets an error, it stops the execution of the rest of the program. An error in Python might be either an error in the syntax of an expression or a Python exception.

Common exceptions in Python

- In Python, there are several built-in Python exceptions that can be raised when an error occurs during the execution of a program. Here are some of the most common types of exceptions in Python:
- **SyntaxError:** This exception is raised when the interpreter encounters a syntax error in the code, such as a misspelled keyword, a missing colon, or an unbalanced parenthesis.
- **TypeError**: This exception is raised when an operation or function is applied to an object of the wrong type, such as adding a string to an integer.
- **NameError**: This exception is raised when a variable or function name is not found in the current scope.
- **IndexError**: This exception is raised when an index is out of range for a list, tuple, or other sequence types.

- **KeyError**: This exception is raised when a key is not found in a dictionary.
- **ValueError**: This exception is raised when a function or method is called with an invalid argument or input, such as trying to convert a string to an integer when the string does not represent a valid integer.
- **AttributeError**: This exception is raised when an attribute or method is not found on an object, such as trying to access a non-existent attribute of a class instance.
- **IOError**: This exception is raised when an I/O operation, such as reading or writing a file, fails due to an input/output error.
- **ZeroDivisionError**: This exception is raised when an attempt is made to divide a number by zero.
- **ImportError**: This exception is raised when an import statement fails to find or load a module.

Exception

- **Def:** An exception is an event, which occurs during the execution of a program that disrupts the normal flow of the program's instructions.
- In general, when a Python script encounters a situation that it cannot cope with, it raises an exception.
- An exception is a Python object that represents an error.
- When a Python script raises an exception, it must either handle the exception immediately otherwise it terminates and quits.
- Exception is the base class for all the exceptions in Python.

Exception handling in Python

- When an error occurs, or exception as we call it, Python will normally stop and generate an error message.
- If you have some suspicious code that may raise an exception, you can defend your program by placing the suspicious code in a **try**: block. After the **try**: block, include an **except**: statement, followed by a block of code which handles the problem as elegantly as possible.
  - The **try**: block contains statements which are susceptible for exception
  - If exception occurs, the program jumps to the **except**: block.
  - If no exception in the **try**: block, the **except**: block is skipped.

Syntax of Exception handling :
try-except-else

```
try:
        # Some Code or do operation here....
except:
        # optional block
        # Handling of exception (if required)
except:
        # optional block
        # Handling of exception (if required)
else:
        # execute if no exception
```

The try clause contains the code that can raise an exception.
The except clause contains the code lines that handle the exception.
The else block lets you execute code when there is no error.

few important points

- A single **try** statement can have multiple except statements. This is useful when the try block contains statements that may throw different types of exceptions.
- You can also provide a generic **except** clause, which handles any exception.
- After the except clause(s), you can include an **else** clause. The code in the **else** block executes if the code in the try block does not raise an exception.
- The **else** block is a good place for code that does not need the try block's protection.

```
#Built in exception raised

n1=int(input("Enter the number:"))
n2=int(input("Enter the number:"))
ans=n1/n2
print("Answer:",ans)
```

Output1:

Enter the number:20
Enter the number:5
Answer: 4.0

Output2:

Enter the number:20
Enter the number:0
Traceback (most recent call last): File "C:/Users/admin/AppData/Local/Programs/Python/Python311/ExceptionDemo.py", line 3, in <module>
    ans=n1/n2
ZeroDivisionError: division by zero

```
#exception handled : Divide by Zero

n1=int(input("Enter the number:"))
n2=int(input("Enter the number:"))
try:
    ans=n1/n2
    print("Answer:",ans)
except:
    print("Sorry...Divide by zero error...")
print("Byee.....")
```

Output1:

Enter the number:20
Enter the number:3
Answer: 6.666666666666667
Byee.....

Output2:

Enter the number:20
Enter the number:0
Sorry...Divide by zero error...
Byee.....

```
# Exception raised before try block at line2

n1=int(input("Enter the number:"))
n2=int(input("Enter the number:"))
try:
    ans=n1+n2
    print("Answer:",ans)
except:
    print("Sorry...Type Error...")
print("Byee.....")
```

Output:

```
Enter the number:3
Enter the number:reshma
Traceback (most recent call last):
  File
"C:/Users/admin/AppData/Local/Programs/
Python/Python311/ExceptionDemo1.py",
line 2, in <module>
    n2=int(input("Enter the number:"))
ValueError: invalid literal for int() with
base 10: 'reshma'
```

```
#So exception defined inside try block

try:
    n1=int(input("Enter the number:"))
    n2=int(input("Enter the number:"))
    ans=n1+n2
    print("Answer:",ans)
except:
    print("Sorry...Type Error...")
print("Byee.....")
```

Output:

```
Enter the number:2
Enter the number:er
Sorry...Type Error...
Byee.....
```

The *except* Clause with No Exceptions

You can also use the except statement with no exceptions defined as follows −

```
try:
        # Some Code or do operation here....
except:
        # optional block
        # Handling of exception (if required)
else:
        # execute if no exception
```

This kind of a **try-except** statement catches all the exceptions that occur. Using this kind of try-except statement is not considered a good programming practice though, because it catches all exceptions but does not make the programmer identify the root cause of the problem that may occur.

```
n1=int(input("Enter the number:"))
n2=int(input("Enter the number:"))
try:
    ans=n1/n2
    print("Answer:",ans)
except:
    print("Sorry...Divide by zero error...")
else:
    print("Nothing went wrong..")
print("Byee.....")
```

Output1:

Enter the number:10
Enter the number:2
Answer: 5.0
Nothing went wrong..
Byee.....

Output2:

Enter the number:10
Enter the number:0
Sorry...Divide by zero error...
Byee.....

```
try:
    n1=int(input("Enter the number:"))
    n2=int(input("Enter the number:"))
    ans=n1/n2
    print("Answer:",ans)
except ZeroDivisionError:
    print("Sorry...Divide by zero error found...")
except ValueError:
    print("Sorry...Value Error found...")
else:
    print("Nothing else went wrong")
print("Byee.....")
```

Output1:

```
Enter the number:10
Enter the number:2
Answer: 5.0
Nothing else went wrong
Byee.....
```

Output2:

```
Enter the number:10
Enter the number:0
Sorry...Divide by zero error found...
Byee.....
```

Output3:

```
Enter the number:10
Enter the number:abc
Sorry...Value Error found...
Byee.....
```

The *except* Clause with Multiple Exceptions

- It is possible to define multiple exceptions with the same except clause. It means that if the Python interpreter finds a matching exception, then it'll execute the code written under except clause.

```
try:
        # Some Code or do operation here....
except(Exception1[, Exception2[,...ExceptionN]]):
        If there is any exception from the given exception list, then execute this
        block
else:
         # execute if no exception
```

- When we define except clause in this way, we expect the same code to throw different exceptions. Also, we want to take the action in each case.

```
try:
    n1=int(input("Enter the number:"))
    n2=int(input("Enter the number:"))
    ans=n1/n2
    print("Answer:",ans)
except (ZeroDivisionError, ValueError)as e:
    print("Error occured: ",e)
print("Byee.....")
```

Output1:

Enter the number:10
Enter the number:0
Error occured: division by zero
Byee.....

Output2:

Enter the number:10
Enter the number:data
Error occured: invalid literal for int() with base 10: ‘data'
Byee.....

```
try:
    n1=int(input("Enter the number:"))
    n2=input("Enter the number:")
    ans=n1+n2
    print("Answer:",ans)
except (ZeroDivisionError)as e:
    print("Error occured: ",e)
except (ValueError)as e:
    print("Error occured: ",e)
except (Exception)as e:
    print("Error occured: ",e)
print("Byee.....")
```

Output1:

Enter the number:10
Enter the number:data
Error occured: unsupported operand type(s) for +: 'int' and 'str'
Byee.....

Output2:

Enter the number:10
Enter the number:10
Error occured: unsupported operand type(s) for +: 'int' and 'str'
Byee.....

Finally Keyword in Python

- Python provides a keyword finally, which is always executed after the try and except blocks.
- The final block always executes after the normal termination of the try block or after the try block terminates due to some exception. The code within the finally block is always executed.

```
try:
        # Some Code....
except:
        # optional block
        # Handling of exception (if required)
else:
        # execute if no exception
finally:
        # Some code .....(always executed)
```

```
n1=int(input("Enter the number:"))
n2=int(input("Enter the number:"))
try:
    ans=n1/n2
    print("Answer:",ans)
except:
    print("Sorry...Divide by zero error...")
finally:
    print("Always executed...")
print("Byee.....“)
```

Output1:

Enter the number:20
Enter the number:2
Answer: 10
Always executed...
Byee.....

Output2:

Enter the number:10
Enter the number:0
Sorry...Divide by zero error...
Always executed...
Byee.....

Advantages of Exception Handling:

- **Improved program reliability**: By handling exceptions properly, you can prevent your program from crashing or producing incorrect results due to unexpected errors or input.
- **Simplified error handling**: Exception handling allows you to separate error handling code from the main program logic, making it easier to read and maintain your code.
- **Cleaner code:** With exception handling, you can avoid using complex conditional statements to check for errors, leading to cleaner and more readable code.
- **Easier debugging**: When an exception is raised, the Python interpreter prints a traceback that shows the exact location where the exception occurred, making it easier to debug your code.

Disadvantages of Exception Handling:

- **Performance overhead:** Exception handling can be slower than using conditional statements to check for errors, as the interpreter has to perform additional work to catch and handle the exception.
- **Increased code complexity**: Exception handling can make your code more complex, especially if you have to handle multiple types of exceptions or implement complex error handling logic.
- **Possible security risks:** Improperly handled exceptions can potentially reveal sensitive information or create security vulnerabilities in your code, so it's important to handle exceptions carefully and avoid exposing too much information about your program.